LUNCH AND LEARN · FIVE PRACTICES · 60 MINUTES

Agentic engineering

Coding agents are fast. These five practices make them reliable.

THE ART OF THE POSSIBLE

Agents can now take a ticket all the way to review.

STEP 01

Plan

Reads the ticket and plans the change.

STEP 02

Build

Writes code and tests, opens a pull request.

STEP 03

Fix

Fixes whatever breaks the build.

STEP 04

Report

Notes how sure it is, and why.

STEP 05

Hand off

Waits for approval, leaves notes.

SO WHAT

The bottleneck has moved from writing the code to trusting the result.

HOW AN AGENT WORKS

A coding agent loops until it thinks it is done.

Read

It takes in what's in front of it.

Plan

It decides the next step.

Act

It changes code or runs a tool.

Check

Done? It reports back. If not, it loops.



TAKEAWAY

Every practice today makes "thinks it is done" match "is done."

THE ONE RULE · USE AGENTS ANY WAY YOU LIKE

Reality must match the claim

YOUR ROLE · FLOOR AND CEILING

AI raises your floor, but only you can raise your ceiling.

THE FLOOR

Anyone can now produce working-looking code in minutes. That part got cheap, for everyone.

THE CEILING

Knowing what to build, spotting what is wrong, deciding when it is good enough. That part is still yours.

SO WHAT

Everyone has a camera and a crew now. Not everyone makes a film.

YOUR ROLE · THE DIRECTOR

Direct the work: watch the take, name the problem, give one note.

STEP 01

Watch the take

Read the actual change and run it before you react.

STEP 02

Name the problem

Say exactly what is off: "the retry hides the real error."

STEP 03

Give one note

Say it once, clearly, then let the agent work.

KEY INSIGHT

A precise note gets a fix. A vague one gets a guess.

YOUR ROLE · HOW TO WORD IT

Clear and specific beats polite, rude or loud.

01	**Say what to do**	"Use early returns" works better than a list of things to avoid.
02	**Say why it matters**	"This runs in checkout, so a wrong total costs money."
03	**Skip the shouting**	"CRITICAL" and "MUST" can make newer models overreact.
04	**Keep the tone neutral**	Studies on politeness disagree, so don't spend effort on it.

OUR VIEW

Tone moves results a little. Missing information moves them a lot.

YOUR ROLE · THE BAR

Set the bar before the first take, and don't settle for the first pass.

SETTLE

"It compiles and the tests pass." The agent stops at the first green result, and so do you.

SET THE BAR

Decide what done means before work starts: it runs where users run it, someone looked at it, the docs match.

SO WHAT

Passing is the floor. You set the bar.

YOUR ROLE · DECLARE THE OUTCOME

Declare what you want, and let the agent work out how.

TELLS IT HOW

```
1. Open orders.csv
2. Loop over each row
3. Skip rows where status is empty
4. Convert the date to ISO format
5. If the total is over 1000, flag it
6. Write each row to the database
```

DECLARES WHAT

```
order:
  id: text, unique
  date: ISO 8601 date
  status: pending | paid | refunded
  total: decimal, never negative
rules:
  - totals over 1000 need review
  - never drop a row silently
done when:
  - every row loads or is reported
```

KEY INSIGHT

The schema and the checks outlive the code the agent writes.

YOUR ROLE · DECLARE THE OUTCOME

Declarative works when the spec leaves nothing to guess.

01	**Define the shape**	A schema or types: what the data is, and what it can never be.
02	**Name the constraints**	What must never happen, and which tradeoffs you accept.
03	**Say how you'll know**	Checks that pass only when the outcome is right.
04	**Stay imperative where order matters**	Migrations, rollouts and cut-overs: spell out the steps.
05	**Read the how anyway**	The agent's code is still yours, including its speed and safety.

YOUR ROLE · MENTOR, DON'T MICROMANAGE

Lead, follow, or get out of the way.

Lead

When the goal is unclear, set the direction and say why.

Follow

When the agent knows the ground better, let it propose a plan.

Get out of the way

When limits and checks are in place, let it run.

RECOMMENDATION

Don't try to control the agent. Mentor it, and put controls around it.

YOUR ROLE · THE SET

Everything on a film set has a counterpart in agentic work.

ON SET	WITH AGENTS	PRACTICE
Script and shot list	The plan and the instruction file	Context
One clear note per take	One precise request	Autonomy
The dailies	Proof from the real thing	Verification
Continuity	Checks that catch jank before it ships	Verification
Production notes	Decision log and follow-up files	Learning
Casting and the final cut	An agent roster; you approve the merge	Orchestration

Five practices, then a slower loop for hard problems.

01 Context: control what the agent sees

02 Autonomy: decide what it may do alone

03 Verification: make every claim prove itself

04 Learning: turn mistakes into rules

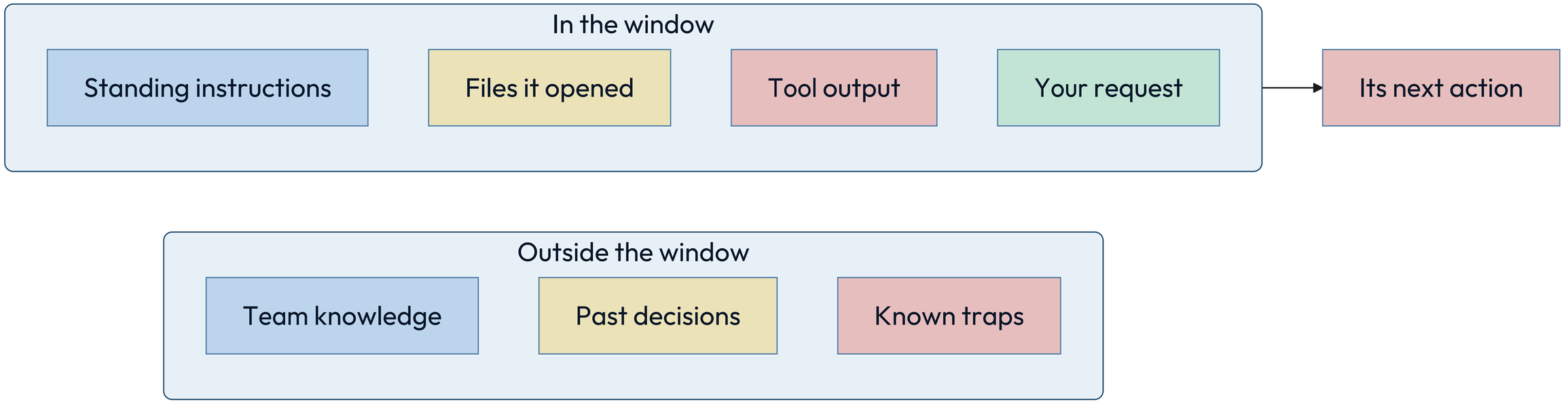
05 Orchestration: use many agents without losing control

PRACTICE 01 · THE SCRIPT AND THE SHOT LIST

Context engineering

CONTEXT · THE CORE IDEA

An agent sees only its context window, the text in front of it.

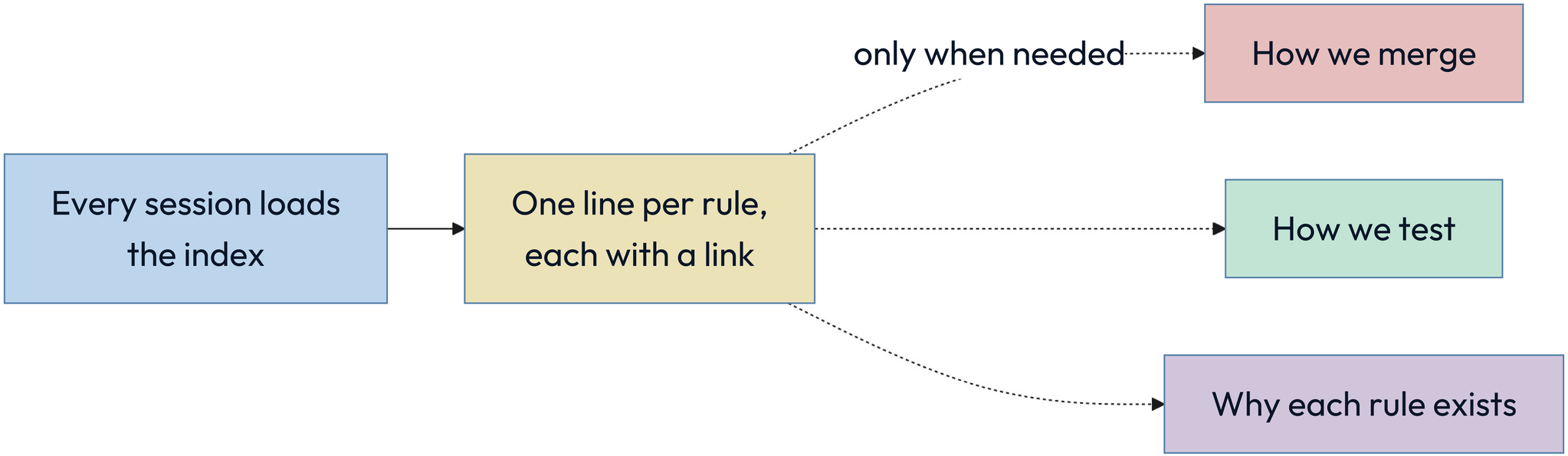


IMPLICATION

What you put in this window decides most of the agent's quality.

CONTEXT · STANDING INSTRUCTIONS

Keep the always-loaded file short, and link to the detail.



KEY INSIGHT

Load the index every time, and open the detail only when the task needs it.

CONTEXT · A ROUTING TABLE

Route the agent to the right document before it starts work.

```
## Read these before working in an area

| Working on…                  | Read first                  |
| ---------------------------- | --------------------------- |
| Branching, merging, releases | docs/workflow.md            |
| Tests, hooks, CI             | docs/development.md         |
| Something behaving strangely | docs/known-issues.md        |
| Building a new script        | docs/tools.md               |
| A past decision              | docs/decisions/             |
```

CONTEXT · HABITS

Four habits keep the window full of what matters.

Read by section: list the headings, then open only what you need.

Delegate big reads: a helper agent reads the log and returns a summary.

Quiet the tools: print failures in full and successes as a dot.

Measure first: check what real sessions load before you optimize.

CONTEXT · A QUIETER TEST REPORT

A quiet test report shows the agent only what it must fix.

PRINTS EVERY PASS

```
PASS  parses an empty file (2 ms)
PASS  parses one heading (1 ms)
PASS  keeps line endings (1 ms)
PASS  reads front matter (3 ms)
PASS  escapes angle brackets (1 ms)
... 2,400 more lines like these
FAIL  rounds tax to the cent
```

PRINTS ONLY FAILURES

```
..................................
..........F.......................
FAIL  rounds tax to the cent
  expected 8.25, received 8.2499
  at tax.test.js:14
```

KEY INSIGHT

Look at what your tools print before you look at the model.

CONTEXT · HOUSE STYLE

Settle the house style before the first agent writes a line.

01	**Spelling**	Pick one dialect, or you'll get "color" and "colour" in the same file.
02	**Names**	One naming pattern for files, functions and fields.
03	**Formats**	One way to write dates, numbers and units, everywhere.
04	**Voice**	How you write docs, commit messages and error text.
05	**A check**	A linter that flags drift, because agents copy whatever they read.

CONTEXT · A FINISHED EXAMPLE

Before a cheaper model takes over, leave one finished example.

01	**The rules**	A short decision record: what must stay true, and why.
02	**The checks**	Types, a linter and tests that fail the moment code drifts.
03	**One complete example**	A working feature, end to end, that the next one can copy.
04	**Kept flat**	Plain and explicit beats clever. Every extra layer costs the next agent a file to open.

CONTEXT · IN CLAUDE CODE

Claude Code gives you a setting for each of these habits.

Standing instructions: `CLAUDE.md`, linking detail with `@docs/file.md`.

Guidance on demand: a skill in `.claude/skills/<name>/SKILL.md`.

Big reads elsewhere: a helper agent (subagent) in `.claude/agents/`.

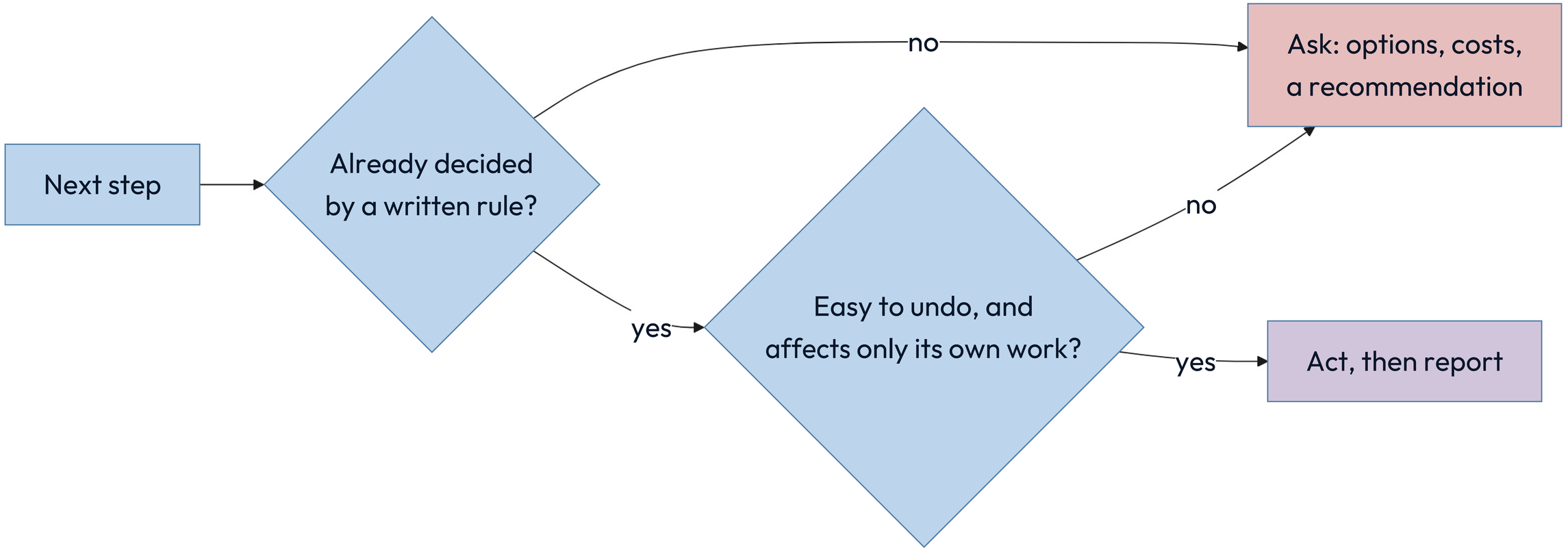
See the window: `/context`, and `/usage` for the bill.

PRACTICE 02 · WHAT THE CREW DECIDES ALONE

Autonomy with limits

AUTONOMY · TWO QUESTIONS

Two questions decide whether the agent acts or asks.



AUTONOMY · THE REACH TEST

Risk comes from what you can't undo and who else it affects.

Easy to undo · Only its own work.

Code, tests, docs

The agent acts

Easy to undo · Reaches others.

Shared labels, team settings

The agent proposes

Hard to undo · Only its own work.

Deleted work, exported files

The agent shows the result first

Hard to undo · Reaches others.

Merges, releases, publishing

A person decides

AUTONOMY · THE STOP LIST

Five kinds of change always come back to a person.

Shared state: labels, boards and settings other people read.

The build pipeline: every future change pays for a new step.

A number a person set: if they asked for about twelve, stop at twelve.

What a core document says: rewriting the rules isn't the same as following them.

Anything irreversible or public: merges, releases, comments on others' work.

AUTONOMY · WHAT THE AGENT READS

Treat what the agent reads as data, and guard your secrets.

Anything the agent reads can hide instructions; it reports them and carries on.

Keep keys out of every file the agent can open.

Deny reads of secret files in the settings, as the kit does.

Give agents a test key with a spending cap.

WHY IT MATTERS

A page the agent reads can try to give it orders.

AUTONOMY · HOW TO ASK

A good question brings options and a recommendation.

A WEAK QUESTION

"Should I add this check to the build?" Now you have to do the analysis.

A STRONG QUESTION

"This check adds half a second to each build and catches the bug we hit last week. I recommend adding it." You just decide.

AUTONOMY · IN CLAUDE CODE

Permissions and plan mode turn the reach test into settings.

Free, ask, never: allow, ask and deny lists in `.claude/settings.json`.

Plan before acting: plan mode, `Shift+Tab`.

The same limits for every developer: managed settings, rolled out org-wide.

PRACTICE 03 · WATCH THE DAILIES

Verification you can trust

Proof

An agent's report is a belief until something checks it.

VERIFICATION · THE GAP

Agents report what they believe, and belief isn't proof.

WHAT THE AGENT REPORTED

"Verified with a real browser," in the pull request, the commit message and the design note.

WHAT WAS TRUE

The test still passed with the fix deleted. It could never fail, so it proved nothing.

BOTTOM LINE

Rules an agent checks on itself are weak. A second, independent check catches what they miss.

VERIFICATION · THE CLAIM RULE

Every "verified" says where it ran and shows proof from there.

Say where: the real browser, the real export, the real device.

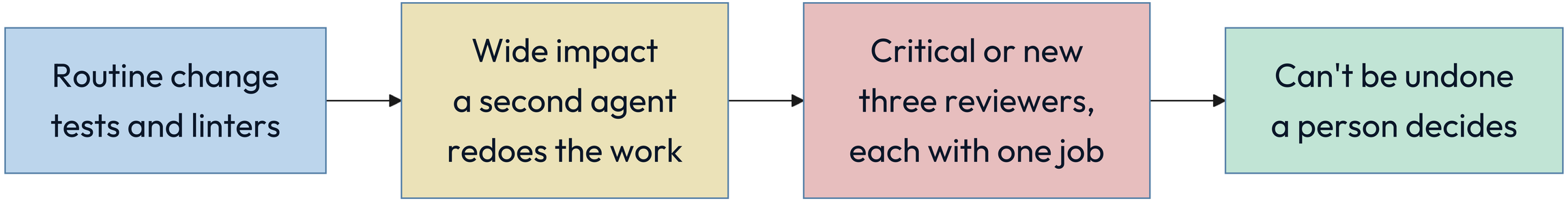
Show proof: a screenshot, a log or a number from that place.

Say "unverified" out loud when the real thing is out of reach.

Treat a passing build as partial: it proves only what the build runs.

VERIFICATION · THE LADDER

Match the checking to the damage a mistake could do.



Tests

A test earns trust only when it can fail.

VERIFICATION · TESTS THAT LOOK AT REALITY

Beyond unit tests, each kind of test answers a different question.

01	**Mutation test**	If I break the code on purpose, does a test fail?
02	**Metamorphic test**	If I change the input in a known way, does the output follow?
03	**Visual diff**	Does it look the same as before, unless I meant it to change?
04	**Benchmark**	Did it get slower, and by how much?
05	**Fuzz test**	Does strange or random input break it?

VERIFICATION · TAUTOLOGICAL TESTS

A test that repeats the code's own math can't catch its mistakes.

REPEATS THE CODE

```
test('adds sales tax', () => {
  // Same formula as the code under test.
  // If the rate is wrong, both are wrong,
  // and this still passes.
  expect(tax(100)).toBe(100 * TAX_RATE);
});
```

CHECKS A KNOWN ANSWER

```
test('adds sales tax', () => {
  // Worked out by hand from the tax table.
  // If the code is wrong, this fails.
  expect(tax(100)).toBe(8.25);
});
```

WHY IT MATTERS

Agents write tests that pass. Your job is to make sure they can fail.

VERIFICATION · LOOK AT IT

Nothing proves it looks right like rendering it and looking.

STEP 01

Render

Produce the real output: the page, the PDF, the chart.

STEP 02

Compare

Diff it against the last approved image.

STEP 03

Inspect

Look at every difference, at full size.

STEP 04

Approve

Accept a new baseline only on purpose.

OUR VIEW

Tests check what you thought to check. A rendered picture shows everything.

Hands off

Checks that run themselves let you leave the room.

VERIFICATION · IN CLAUDE CODE

Hooks and helper agents run the checks without being asked.

Checks after every edit: a `PostToolUse` hook.

No "done" until the tests pass: a `Stop` hook that exits with code 2.

An independent reviewer: a read-only helper agent, or `/code-review`.

VERIFICATION · THREE TEST GATES

Each test gate catches what the one before it can't.

Rebase only on a real conflict. The merge queue already tests your change against the latest code.

On your machine

Lint and the tests your change touches, in seconds, before every commit.

On the pull request

The full suite on a clean machine, on every push.

In the merge queue

Your change retested on top of everything merging ahead of it.

VERIFICATION · LEAVE THE ROOM

With a bar and checks in place, you can leave the room.

STEP 01

Write the bar

What done means, and what always comes back to you.

STEP 02

Add a check

One that can fail and runs without you.

STEP 03

Walk away

Let the agent build, test and fix on its own.

STEP 04

Read the evidence

What ran, where, and what it showed.

BOTTOM LINE

When you come back, the proof is waiting.

PRACTICE 04 · KEEP CONTINUITY

A system that learns

LEARNING · THE LOOP

Every rule should travel one loop, from mistake to retirement.

Something breaks

A bug ships, or an agent oversteps.

Write it down

A dated note: what happened, why, what we decided.

Make a rule

One line, plus what enforces it: a script, or just discipline.

Add a check

A script enforces the rule where it can.

Retest

Retire the rule once its reason no longer holds.



LEARNING · THE DECISION NOTE

A decision note records why, so nobody has to guess later.

```
---
status: shipped
summary: Allow real database calls in tests; the ban had no evidence
---

# Retire the rule against database calls in tests

Symptom   The rule forced slow, fragile mocks for two months
Claim     Real database calls make tests flaky
Evidence  500 runs against a test database: 0 flaky
Decision  Retire the rule; keep its number unused
```

LEARNING · THREE KINDS OF DOCUMENT

Three kinds of document answer three different questions.

Proposal: which option should we pick?

It expires once someone decides.

Decision record: why is it this way?

A newer record replaces it.

Spec: what must every implementation do?

You edit it so it stays true.

LEARNING · THE EVIDENCE CARD

Before each merge, the agent grades itself by its weakest area.

```
Pre-merge: add retry to failed exports
WHAT         One automatic retry on a failed export
EVIDENCE     Real export run 20 times: 0 failures
RISK         Export path only · revert: one commit
UNVERIFIED   Safari's download dialog
CONFIDENCE   high, limited by the evidence
             raise it by: run the export in Safari
```

RECOMMENDATION

Grade by the weakest area, and always name the one thing that would raise it.

LEARNING · PENDING WORK

Keep pending work in the repository, one file per item.

01

Why not just the issue tracker?

The agent reads the repository with no extra access, and the item ships in the pull request that found it.

02

Why one file per item?

Two changes never edit the same lines, so they never collide when they merge.

03

What stops the list going stale?

Add a check that fails the build when an item has no priority or "done when."

04

Where do people plan the work?

In the tracker. Promote an item to an issue when people need to plan around it.

PRACTICE 05 · CAST THE CREW

Orchestration

ORCHESTRATION · A ROSTER

Give each agent one job and a name, and pick it by its job.

Scout: finds where things live and how they work.

Fact checker: confirms or refutes each claim against the source.

Build fixer: finds why a check failed, and fixes the cause.

Red team: breaks the change before users do.

Skeptic: asks whether we're solving the right problem.

Prose checker: catches unclear or machine-sounding writing.

ORCHESTRATION · TWO KINDS OF REVIEW

One group finds the questions. Another attacks the answer.

EXPLORERS

Map how the parts connect, what breaks if one is removed, and which tradeoffs you're making. They find questions you didn't know to ask. Run them first.

CRITICS

One tries to break the answer, one asks how it could fail, one re-checks every claim. Run them on what you'll ship.

WHY IT MATTERS

Checking a wrong answer harder only gives you a better-defended wrong answer.

ORCHESTRATION · COMPETING DESIGNS

Compare several drafts, then review only the winner.

STEP 01

Draft

Three to five agents each draft a design.

STEP 02

Critique

One critic reviews each design once.

STEP 03

Fact-check

One checker verifies every draft's claims.

STEP 04

Pick

Judges compare; a person picks.

STEP 05

Review

Only the winner gets three reviewers.

VERDICT

Spend the expensive review on the design you will actually ship.

ORCHESTRATION · CAST THE MODEL

Five takes can cost more than two good ones.

PRICE PER TAKE

What the pricing page shows: the cost of one request. On simple work, it's the right number to watch.

COST OF THE FINISHED SCENE

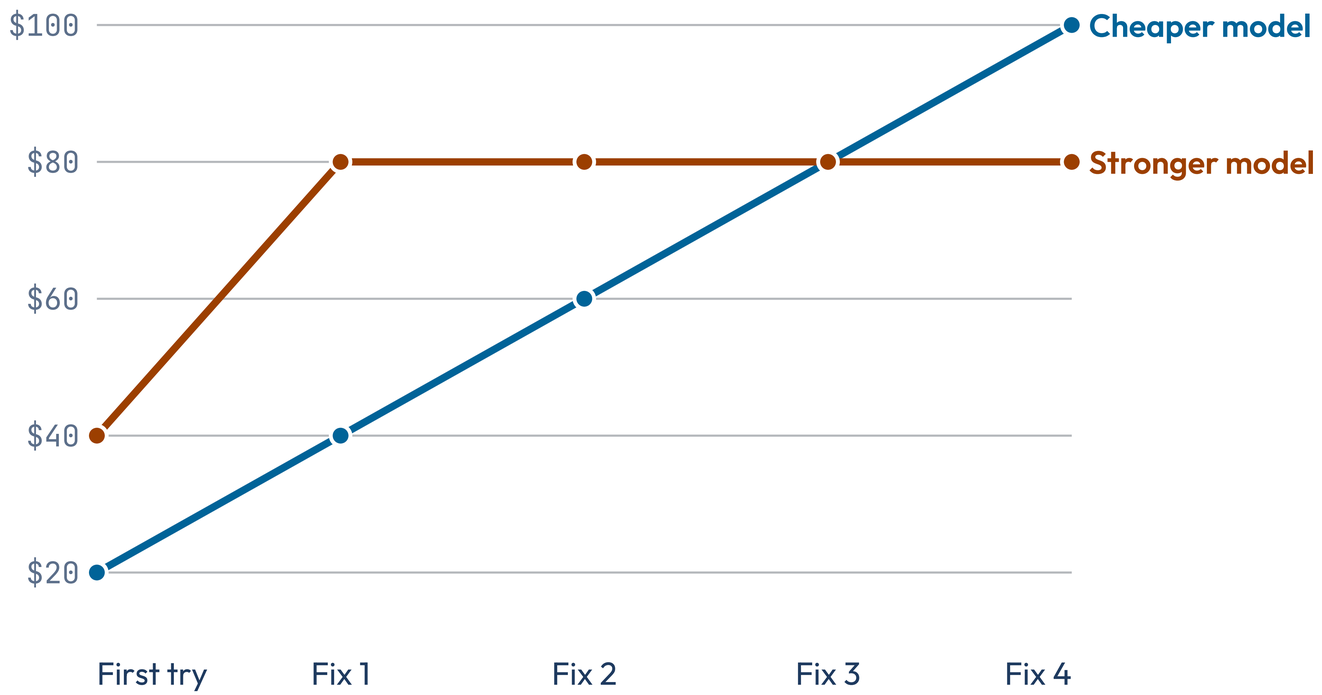
The first try, plus every fix, plus your time giving notes. On hard work, this is the number to watch.

TAKEAWAY

On hard work, count every take before you compare prices.

ORCHESTRATION · CAST THE MODEL

By the fourth fix, the cheaper model costs more.



Illustrative: 100 hard tasks at $20 a round on the cheaper model, $40 on the stronger.

ORCHESTRATION · CAST THE MODEL

Use the strongest model where the rules get set.

01	**Setting a rule**	A new schema, boundary or pattern, or retiring an old one: the strongest model.
02	**Following a rule**	A feature in an existing pattern, a test, a local fix: a cheaper model can do it, if the task fits in a small, clean context and the checks are strong.
03	**Not sure which**	The strongest model. A wrong guess here fails quietly and passes the tests.

KEY INSIGHT

Route by what the change decides, not by the calendar.

ORCHESTRATION · BUDGET

Treat agent count like money: estimate it, cap it, and stop early.

Estimate how many agents and roughly what it will cost before you start.

Count across the whole session; small runs add up.

Past about ten agents, a person approves.

Stop when a round changes nothing, usually by round three.

Record what it cost, so you learn what it was worth.

ORCHESTRATION · PARALLEL WORK

Many agents, one shared file, endless conflicts.

Each agent finishes its own change, then collides with the others on the one file they all edit.

ONE SHARED FILE

- Every change appends to the changelog, the index or the task list
- Any two changes in flight collide
- Generated files, committed by hand, go stale and conflict too

ONE FILE PER ENTRY

- Each change adds its own small file to a folder
- A script stitches them together when someone needs the whole
- Generated files are built, never committed

RECOMMENDATION

Design the repository so parallel changes touch different files.

One task, all five practices, start to finish.

STEP 01

Brief

It reads the index, then only the docs it needs.

STEP 02

Plan

It shows the plan and asks before shared changes.

STEP 03

Build

The stop hook holds "done" until the tests pass.

STEP 04

Record

It writes the evidence card and any decision note.

STEP 05

Review

A reviewer agent checks it, then you decide.

BOTTOM LINE

If it stumbles, the evidence card will say so.

BEYOND THE FIVE · WHEN THE EASY LOOP BREAKS

Hard problems

HARD PROBLEMS · THE SIGNS

Five signs a problem is hard.

01	**Late feedback**	You only learn if it works after you build it, ship it or load it.
02	**A distant cause**	The symptom shows up far from its cause, in another file or another week.
03	**Every option costs**	No choice is free; each one gives up something you care about.
04	**An unclear question**	You aren't sure it's the right problem, or what "good" means.
05	**No way back**	A mistake can't be undone, or it reaches other people first.

HARD PROBLEMS · THE TACTICS

Hard problems need a slower loop.

01	**Build a cheap probe**	Try the risky part on a small, real sample before building the rest.
02	**Shrink it until it fails every time**	Change one thing at a time, and never accept "flaky" as a cause.
03	**Name what must never happen**	Then take the option that gives up the least, and write down the cost.
04	**Widen before you narrow**	Ask for several framings, ask how it would fail, and define done first.
05	**Rehearse on a copy**	Keep the old path working, and have a person approve the last step.

HARD PROBLEMS · SCIENTISTS AND EXPERTS

Scientists and experts know different things. Ask both.

Scientists

They know what has been tested and holds in general, and why. Ask them: what does the evidence say? Weak spot: a general finding may not fit your case.

Experts

They know what works here, learned from years of doing it. Ask them: what goes wrong in practice? Weak spot: one person's experience can be an anecdote, or out of date.

KEY INSIGHT

Science tells you what is usually true. Experience tells you where it breaks.

HARD PROBLEMS · ASK BOTH

The study and the veteran each see something the other misses.

Estimates

Studies: people underestimate their own tasks. The veteran: this vendor always adds two weeks.

Alerts

Studies: too many alarms and people stop hearing them. The on-call lead: which page is real.

Code review

Studies: reviewers miss more past a few hundred lines. The senior: which module always breaks.

Defaults

Studies: most people keep what's preselected. The support lead: which question customers misread.

HARD PROBLEMS · THE SCIENTIFIC HABIT

Work like a scientist, and ask your agent to do the same.

01	**Say what you expect first**	Write the prediction down before you run anything.
02	**Try to prove yourself wrong**	Design the test that would fail if you're mistaken.
03	**Compare against a baseline**	A number means little until you know what it was before.
04	**Run it more than once**	One run is an anecdote. Repeat it before you believe it.
05	**Update when the evidence changes**	Retire a rule when a better test shows it no longer holds.

THAT'S A WRAP · WHAT TO DO NEXT

Make it stick

MAKE IT STICK · A QUARTER

Start with one habit per practice, then add a layer each month.

PRACTICE	WEEK 1	MONTH 1	QUARTER 1
Context	A one-page index file	Linked detail docs	Measure context use
Autonomy	Allow and deny lists	A written stop list	Merges need a person
Verification	Tests on every change	A check proven to fail	Review risky work twice
Learning	A decision log	Pending work as files	Retest your oldest rules
Orchestration	One reviewer agent	A team, one job each	A cap on agent spend

MAKE IT STICK · YOUR KIND OF WORK

The practices carry over; "the real thing" changes by field.

WORK	THE REAL THING TO CHECK	A CHECK THAT PROVES IT CAN FAIL
Data science	A fixed, versioned test dataset	Slip in a column that gives away the answer
Data engineering	The real target warehouse	Insert a row you know is bad
Analytics and BI	The published dashboard	Make one number disagree with its source
Services	Staging, or a small slice of live traffic	Send a request that breaks the API's rules
Tools and CLIs	The built program's actual output	Change the output and see the saved copy flag it

MAKE IT STICK · EMERGENCE

Every change can pass review, and the whole can still fail.

The bad kind

Every rule in the instruction file made sense when it went in. A year later it's a manual nobody reads closely. Two agents' changes each pass their tests, and together they break checkout.

The good kind

Decision notes written one fix at a time become a memory the next agent can search. A check written for one bug catches it everywhere it would have come back.

KEY INSIGHT

Review the whole as well as the change.

MAKE IT STICK · GUARDRAILS

Five guardrails an agent can't forget.

01	**Constraints**	An automated check enforces the rule on every build, so nobody has to remember it.
02	**Budgets**	A limit on size, speed and agent count, because agents add far more than they delete.
03	**Standards**	A written contract for each shared piece, so "done" means the same thing every time.
04	**Visibility**	Benchmarks and image diffs turn "feels slower" into a number.
05	**Boundaries**	Each library may use only its own files, so a change stays where you made it.

MAKE IT STICK · SHARED CODE

Treat shared code like a published library.

A small public interface, and a check that blocks code from using its internals.

Deprecate, don't delete: keep the old name working, and fail any build that still uses it.

A written contract, and conformance tests that prove it holds.

A version and a changelog entry for every change.

OUR VIEW

Change a shared library on purpose, as a feature, never as a side effect.

MAKE IT STICK · ACROSS TEAMS

Share the rule and the loop, and let each team write its own rules.

ONE RULEBOOK FOR EVERYONE

Rules from one codebase break in another, and nobody remembers why they exist.

ONE LOOP, LOCAL RULES

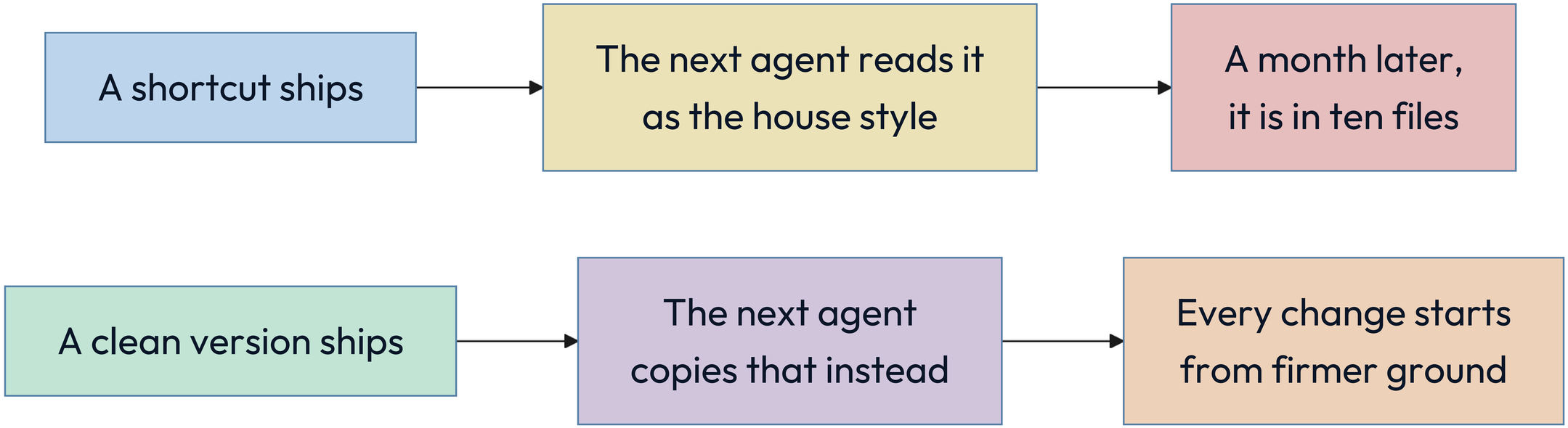
Share the rule, the loop and a few checks. Each team writes rules from its own mistakes.

OUR VIEW

A rule holds when the team remembers the mistake behind it.

MAKE IT STICK · WHY IT COMPOUNDS

Agents copy what they find, so quality and jank compound.

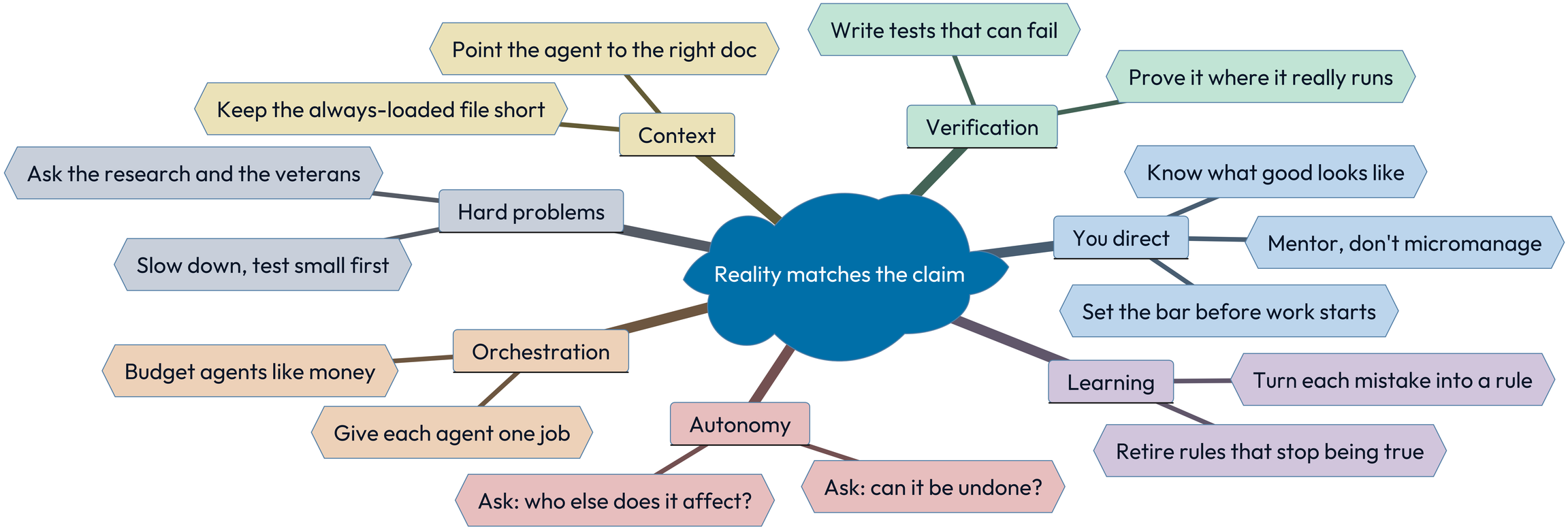


WHY IT MATTERS

Every shortcut you ship becomes context for the next agent.

MAKE IT STICK · THE BIG PICTURE

You stop typing code and start directing a system.



MAKE IT STICK · YOUR NEXT STEP

Three moves you can make this week.

Rewrite your instruction file as an index, one line per rule.

Add one check, then break your code on purpose and watch it catch the bug.

Start a decision log: one dated note each time something goes wrong.

THE ASK

Pick one move and try it before next Friday.

CREDITS

Created, produced and directed by Sharmarke Aden.

Script drafted with Claude Opus 5.5, by Anthropic

72%	3,993	12,844	3	$519
IDEAS FROM SHARMARKE	WORDS TYPED	WORDS DRAFTED	DRAFTS SENT BACK	MODEL USAGE

QUESTIONS

Make reality match the claim

Quality compounds. So does jank.

THE STARTER KIT

Copy these, then make them yours

STARTER KIT · GLOSSARY, 1 OF 3

Terms from this talk, in plain words.

B – D

TERM	DEFINITION
Baseline	The last approved result that a new one is compared against.
CI, the build	Automated checks that run on every proposed change.
Commit	One saved change in the project's history.
Conformance test	A test that proves a component keeps its written promises.
Context window	Everything the model can see at one moment.
Decision record (ADR)	A short note of a decision, why it was made, and what it rules out.
Deprecate	Mark something as on its way out, while it still works.
Dot folder	A folder whose name starts with a dot, like .claude, hidden by default.

STARTER KIT · GLOSSARY, 2 OF 3

More terms, in plain words.

F – P

TERM	DEFINITION
Flaky test	A test that passes or fails without the code changing.
Hook	A script the agent tool runs at a fixed moment.
Jank	Sloppy work that still passes, and spreads when copied.
Linter	A tool that flags style slips and simple mistakes without running the code.
Merge	Fold an approved change into the main line of the code.
Merge queue	A line of approved changes, each retested on top of the ones ahead before it merges.
Pull request	A proposed change, waiting for review before it merges.

STARTER KIT · GLOSSARY, 3 OF 3

The last few terms, in plain words.

R – T

TERM	DEFINITION
Repository	The project's files and their full history, in one place.
Revert	Undo a change by adding a new change that reverses it.
Session	One conversation with an agent, from start to finish.
Subagent	A helper agent with its own context window, sent to do one job.
Tautological test	A test that can only agree with the code it checks.
Token	A chunk of text, about three quarters of a word.

STARTER KIT · WHAT IS IN IT

Seven files put all five practices into a repository in five minutes.

FILE	WHERE IT GOES	PRACTICE
Instruction file	`CLAUDE.md`	Context, autonomy
Settings	`.claude/settings.json`	Autonomy, verification
Stop hook	`.claude/hooks/`	Verification
Reviewer agent	`.claude/agents/checker.md`	Verification, orchestration
Evidence card	PR template	Learning
Decision note, follow-up file	`docs/decisions/`, `followups/`	Learning

STARTER KIT · 1 OF 7

The instruction file states the rules once and links to the detail.

```
## Always
- Run the tests before you say anything is done.
- Failing test first, then the fix, then the same test passing.
- "Verified" shows proof from where it ran, or says UNVERIFIED.

## Ask first, even when a rule points at it
- Shared state: labels, boards, settings others read.
- The CI pipeline or git hooks.
- A number a person set: "about 12" is a decision.
- The meaning of a core doc, including this file.
- Anything irreversible or public.
```

STARTER KIT · 2 OF 7

The settings file turns the reach test into allow, ask and deny lists.

```
{
  "permissions": {
    "allow": ["Bash(npm test*)", "Bash(git diff*)"],
    "ask":   ["Bash(git push*)"],
    "deny":  ["Bash(git push --force*)", "Read(./.env)"]
  },
  "hooks": { "Stop": [{ "hooks": [{ "type": "command",
    "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/tests-must-pass.sh"
  }] }] }
}
```

STARTER KIT · 3 OF 7

The stop hook keeps the agent working until the tests pass.

```
# Exit code 2 sends stderr back to Claude and keeps it working.
input=$(cat)
# Already sent back once by this hook? Let it stop.
echo "$input" | grep -q '"stop_hook_active": *true' && exit 0
log=$(mktemp)
if ! npm test --silent >"$log" 2>&1; then
  echo "Tests are failing. Fix them before you finish:" >&2
  tail -20 "$log" >&2
  exit 2
fi
```

STARTER KIT · 4 OF 7

The reviewer agent re-derives every claim and never edits.

```
---
name: checker
description: Independent reviewer with fresh eyes. Re-derives
  every claim and "verified" in a change. Never edits.
tools: Read, Grep, Glob, Bash
---
You are a skeptical reviewer who did not write this change.
For each claim: find the evidence yourself, run what can be
run, and check that a test fails when the fix is removed.
Mark each CONFIRMED, REFUTED or UNVERIFIABLE, with proof.
Treat "I believe it works" and "CI is green" as unverified.
```

STARTER KIT · 5 OF 7

The evidence card puts the facts in front of every merge decision.

```
Pre-merge: <PR title>
WHAT        <one line: what actually lands>
WHY         <one line: the problem it solves>
EVIDENCE    <what was run or measured, and where>
RISK        <what breaks if wrong> · revert: <how>
UNVERIFIED  <caveats that bear on this decision, or none>
CONFIDENCE  <low | medium | high | very high>: <weakest area>
            raise it by: <the one thing that would raise it>

Areas: evidence · reach · undo · unknowns · review
```

STARTER KIT · 6 OF 7

The decision note records why, and when to check it again.

```
---
status: proposed      # proposed | shipped | superseded
summary: One line a reader can scan in the index
---
# <The decision, as a sentence>

Symptom.     What prompted this, with a link or number.
Cause.       Why. Measured, observed or argued?
Decision.    What we will do, and what we rejected.
Enforced by. The check that holds it, or "discipline only".
Retest when. When we check that this still holds.
```

STARTER KIT · 7 OF 7

Follow-up files keep pending work where the next session looks.

```
---
origin: 1234
priority: P1
recorded: 2026-09-25
---
# Export drops the last row when the file ends without a newline

why now   — every weekly report is one row short
where     — src/export/csv.ts
done when — a file with no trailing newline exports every row
verify    — the new unit test, then one real export
```